

EMBEDDED COVER DEMO

# Markdown Studio

## Local PDF Export

### Feature Demo PDF

Cover, PDF index links, bookmarks, diagrams, math, and local assets.

---

Source: embedded Markdown cover block

# Table of Contents


Markdown Studio — Feature Demo p.3
- 1. Markdown Rendering p.3
  - 日本語テスト p.3
  - Task Lists p.3
  - Footnotes p.4
  - Emoji p.4
  - Math (KaTeX) p.4
  - Definition Lists p.4
  - Superscript / Subscript p.4
- 2. Mermaid Diagrams p.5
  - Markdown Studio Architecture p.5
  - Extension Activation Flow p.6
- 3. PlantUML Diagrams p.6
  - Extension Component Diagram p.7
  - Document Processing Sequence p.8
- 4. Inline SVG (Sanitized) p.8
- 5. Syntax Highlighting p.8
- 6. Images and Security p.10
  - ✅ Local image (relative path — works) p.10
  - ✅ Local image (absolute path — works) p.11
  - ❌ External image (blocked by policy) p.11
  - ❌ External link (blocked by policy) p.11
  - How to allow external resources p.11
  - Security Summary p.12
- 7. Theme Adaptability p.12
- 8. Diagram Type Catalog p.12
  - WaveDrom Timing Diagram p.19
  - Wide Diagrams for PDF Aspect Ratio p.19
- 9. Custom CSS p.20
  - Bundled Themes p.20
  - Setup p.20
  - How It Works p.21
- 10. PDF Export p.21

# Markdown Studio — Feature Demo

Open this file and run **Markdown Studio: Preview** ( `Cmd+Shift+P` ).

---

## 1. Markdown Rendering

**Bold**, *italic*, ~~strikethrough~~, `inline code`

> Blockquotes work too.

| Feature | Status | Remark |
|---|---|---|
| Markdown | ✅ | A |
| Mermaid | ✅ | B |
| PlantUML | ✅ | C |
| SVG | ✅ | D |
| WaveDrom | ✅ | E |
| PDF Export | ✅ | F |

1. Ordered list item
2. Another item
3. Third item

- Unordered item
- Another item

- Item

### 日本語テスト

- 日本語あ `いう` えお
    - 2byte文字 `テスト`

### Task Lists

- [x] Markdown rendering
- [x] Mermaid diagrams
- [x] PlantUML diagrams
- [x] WaveDrom timing diagrams

- [x] LaTeX math
- [x] Footnotes
- [ ] no checked

## Footnotes

This sentence has a footnote[1] and another one[2].

## Emoji

😄 🚀 👍 ⚠️ ❤️ ⭐ 🔥 ☕

## Math (KaTeX)

Inline math: $E = mc^2$ and $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$

Display math:

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$$

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

## Definition Lists

**Markdown**
: A lightweight markup language for creating formatted text.

**PlantUML**
: A tool for creating UML diagrams from plain text.

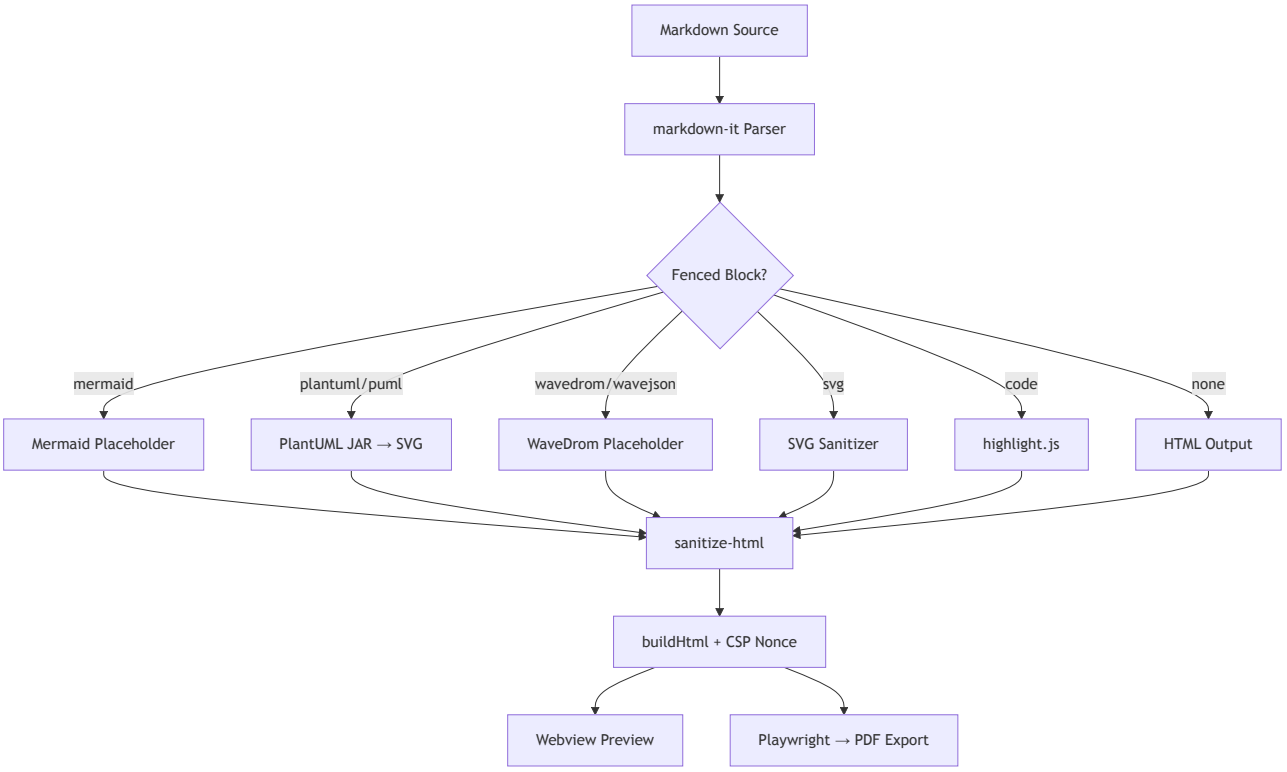
  Uses Smetana layout engine (no Graphviz needed).

## Superscript / Subscript

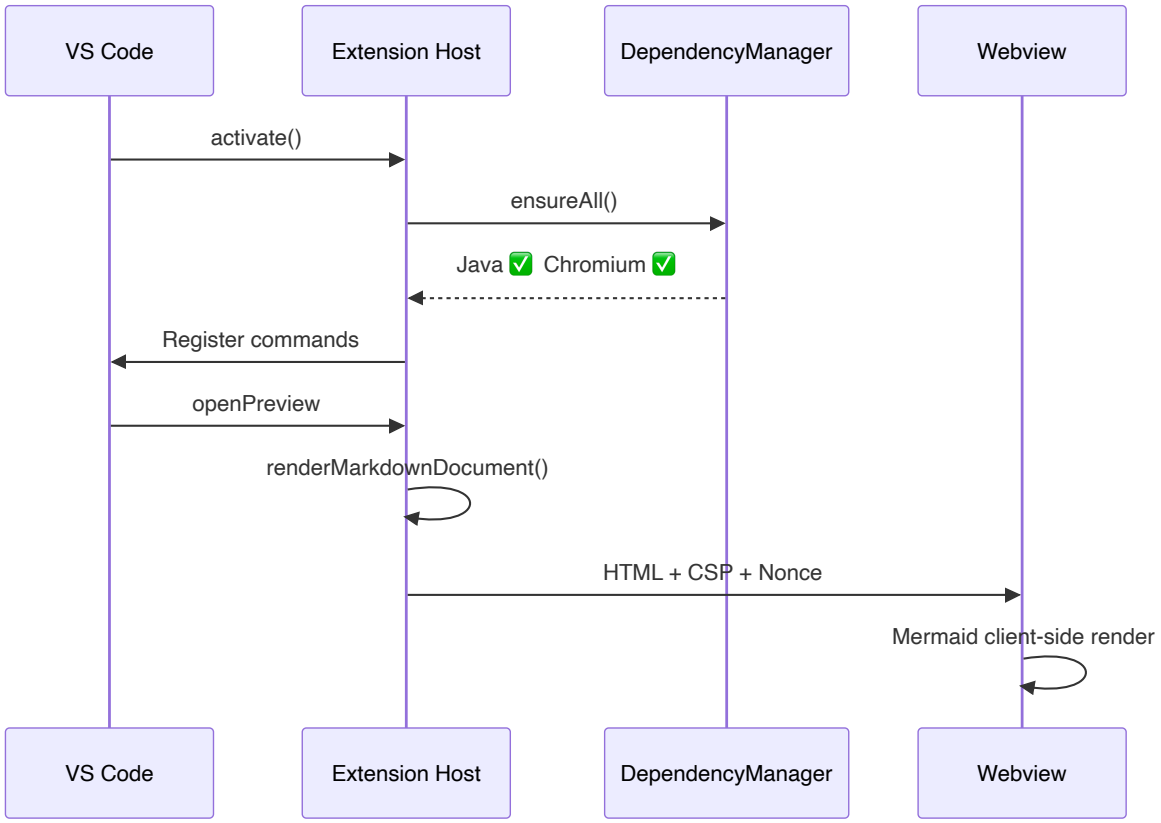
- Water: $H_2O$
- Einstein: $E = mc^2$
- $19^{th}$ century

# 2. Mermaid Diagrams

## Markdown Studio Architecture



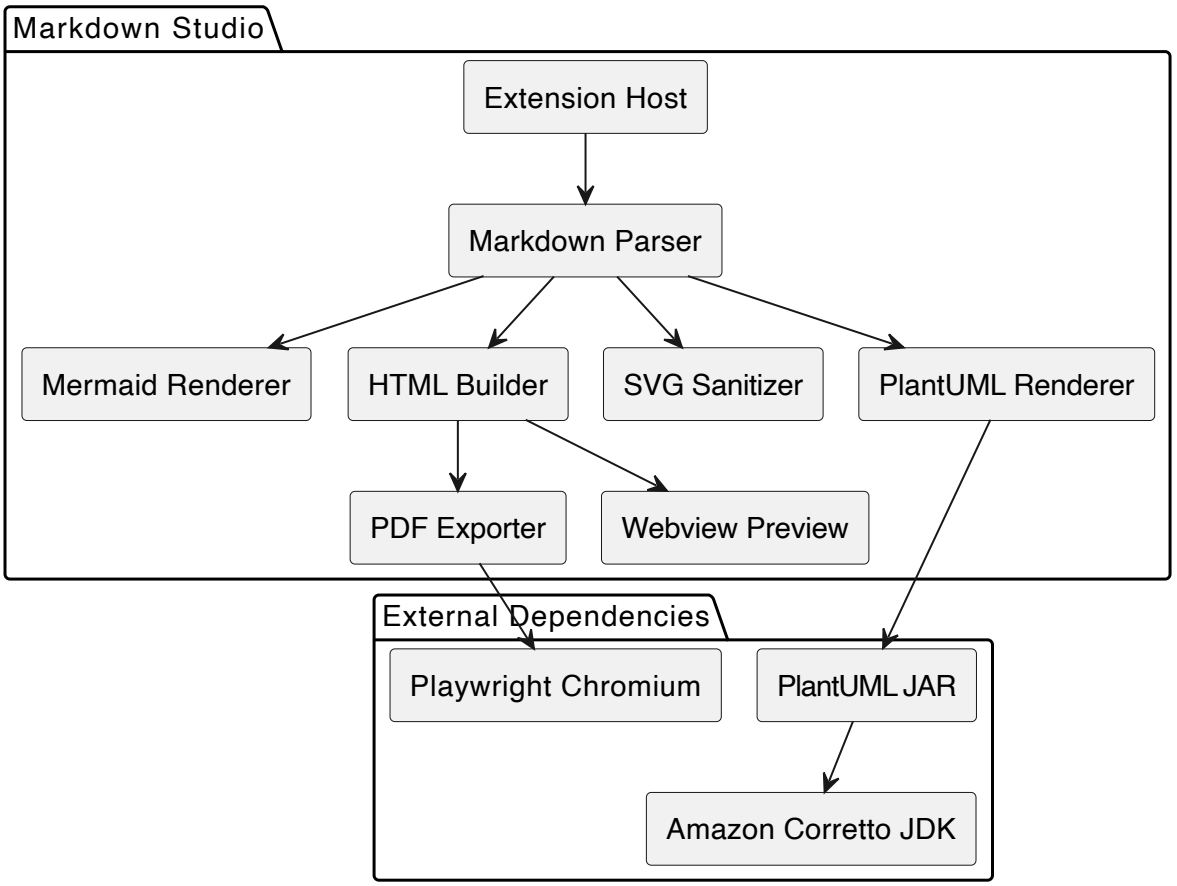
### Extension Activation Flow



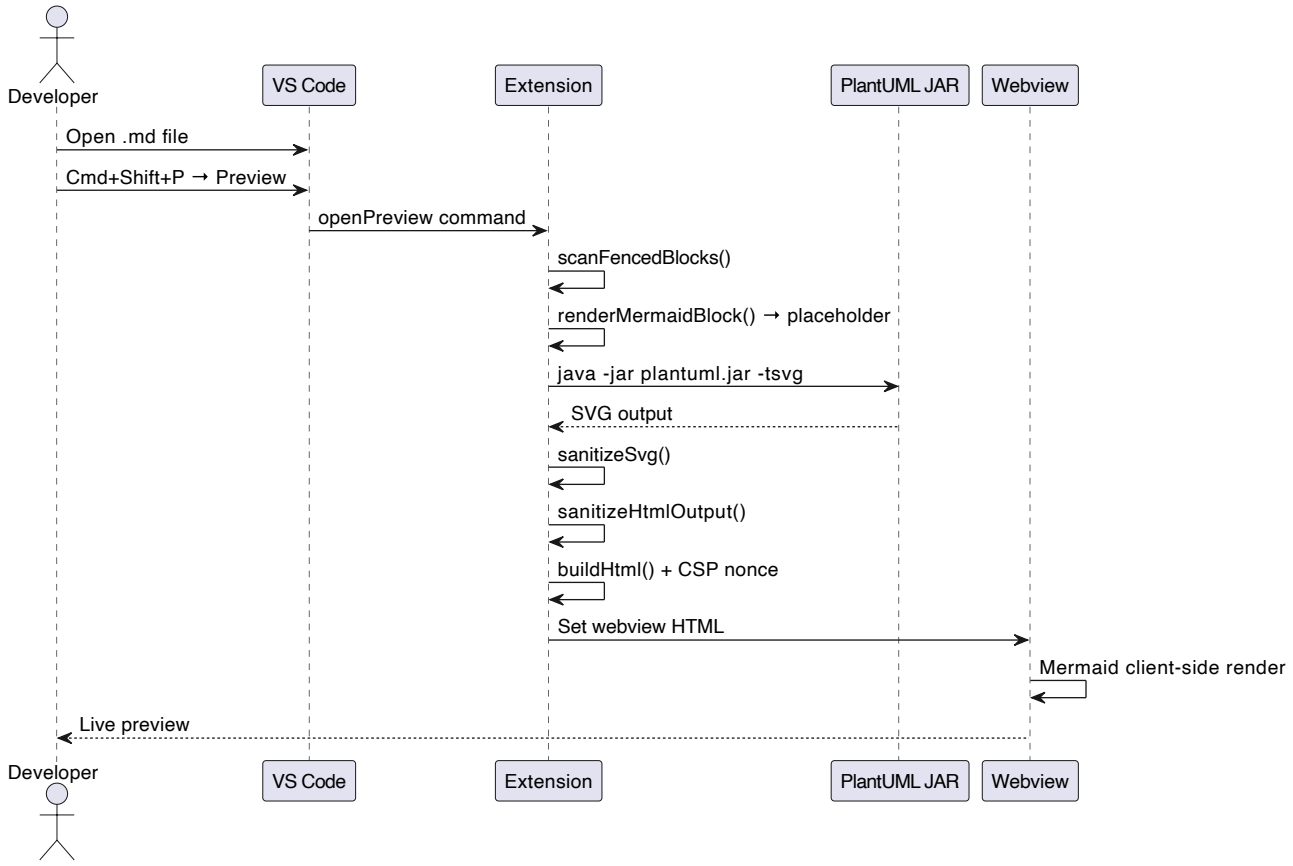

## 3. PlantUML Diagrams

Rendered locally via bundled JAR. No remote server.

### Extension Component Diagram



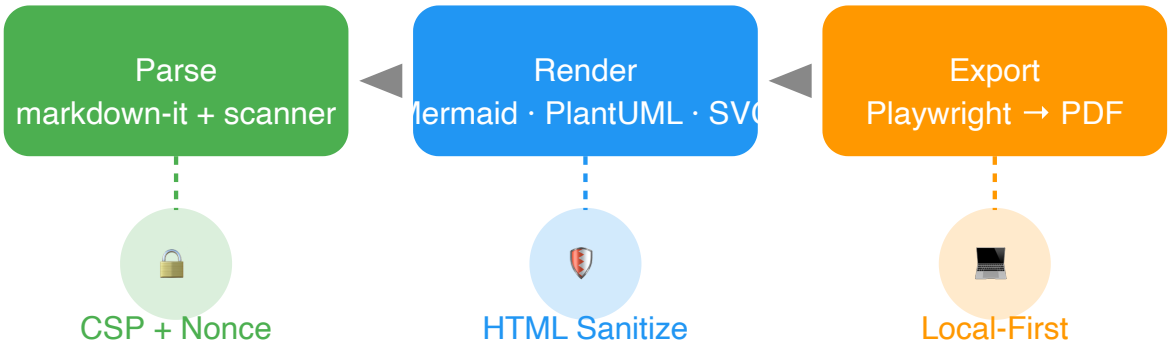
### Document Processing Sequence




---

## 4. Inline SVG (Sanitized)

Dangerous elements ( `<script>` , `<foreignObject>` , event handlers) are stripped automatically.




---

## 5. Syntax Highlighting

```
import * as vscode from 'vscode';

export function activate(context: vscode.ExtensionContext): void {
  const depManager = new DependencyManager();
  const status = await depManager.ensureAll(context);

  context.subscriptions.push(
    vscode.commands.registerCommand('markdownStudio.openPreview', async () => {
      await openPreviewCommand(context);
    }),
    vscode.commands.registerCommand('markdownStudio.exportPdf', async () => {
      await exportPdfCommand(context);
    })
  );
}
```

```
{
  "markdownStudio.plantuml.mode": "bundled-jar",
  "markdownStudio.java.path": "java",
  "markdownStudio.export.pageFormat": "A4",
  "markdownStudio.security.externalResources.mode": "whitelist"
}
```

```
# PlantUML rendering is also useful for Python projects
from dataclasses import dataclass

@dataclass
class DiagramConfig:
    mode: str = "bundled-jar"
    java_path: str = "java"
    timeout_ms: int = 15000
```

```
fmt.Println("Hello, World!")
```

```

```

```
plain text without language
```

```
fn main() {
    println!("Hello from Rust");
}
```

```
SELECT id, name FROM users WHERE active = true;
```

```
FROM node:20-alpine
WORKDIR /app
COPY . .
RUN npm install
```

## 6. Images and Security

Markdown Studio is local-first. The default security policy ( `externalResources.mode = "whitelist"` ) allows local files and selected trusted remote domains.

### ✅ Local image (relative path — works)

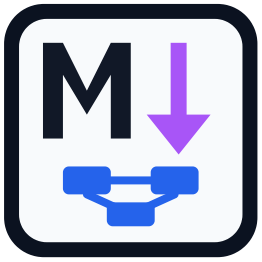
- local svg




- icon.png



- icon.svg



Relative paths from the Markdown file are resolved and displayed correctly.

## ✅ Local image (absolute path — works)

Images with absolute file paths also work in the preview.

## ❌ External image (blocked by policy)

> External image blocked by policy.

External URLs outside the allowlist are blocked by default. The image above shows a policy notice instead.

## ❌ External link (blocked by policy)

External link — blocked by default ✋

External links outside the allowlist are replaced with a blocked notice in the preview.

## How to allow external resources

Set `markdownStudio.security.externalResources.mode` to `allow-all` in VS Code settings to allow external images and links. This is not recommended for corporate/secure environments.

### Security Summary

| Resource Type | Default | Configurable |
|---|---|---|
| Local images (relative path) | ✅ Allowed | — |
| Local images (absolute path) | ✅ Allowed | — |
| External images (https://) | ❌ Blocked unless allowlisted | `externalResources.mode: allow-all` |
| External links (https://) | ❌ Blocked unless allowlisted | `externalResources.mode: allow-all` |
| Inline SVG | ✅ Allowed | — |
| Mermaid diagrams | ✅ Allowed | — |
| PlantUML diagrams | ✅ Allowed | — |
| WaveDrom diagrams | ✅ Allowed | — |

---

## 7. Theme Adaptability

Switch between **light** and **dark** mode in VS Code ( `Cmd+K Cmd+T` ) to see how the preview adapts:

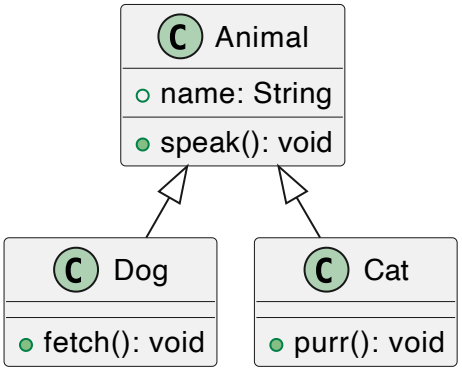
- Mermaid diagrams automatically switch between light and dark themes
- WaveDrom diagrams keep a light timing-diagram surface for readability
- SVG elements use colors chosen for visibility in both themes
- Code blocks use theme-aware syntax highlighting
- PlantUML output receives CSS overrides for dark mode

---

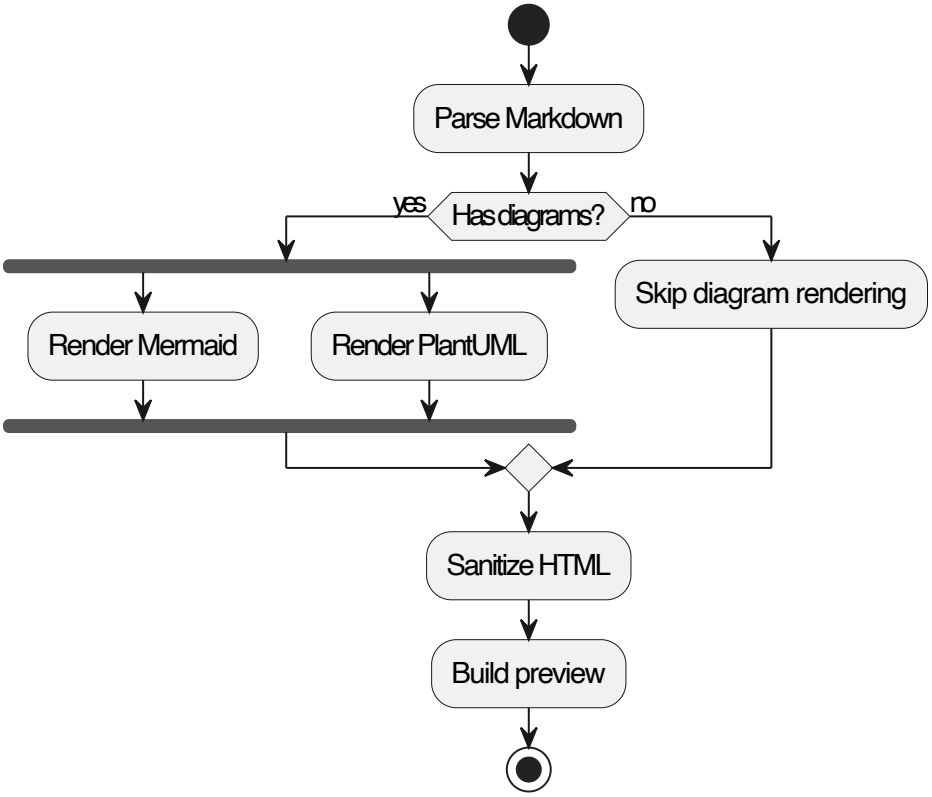
## 8. Diagram Type Catalog

All diagram types below are verified to render locally. PlantUML uses the bundled Smetana engine, Mermaid and WaveDrom render in the webview runtime, and inline SVG is passed through directly.

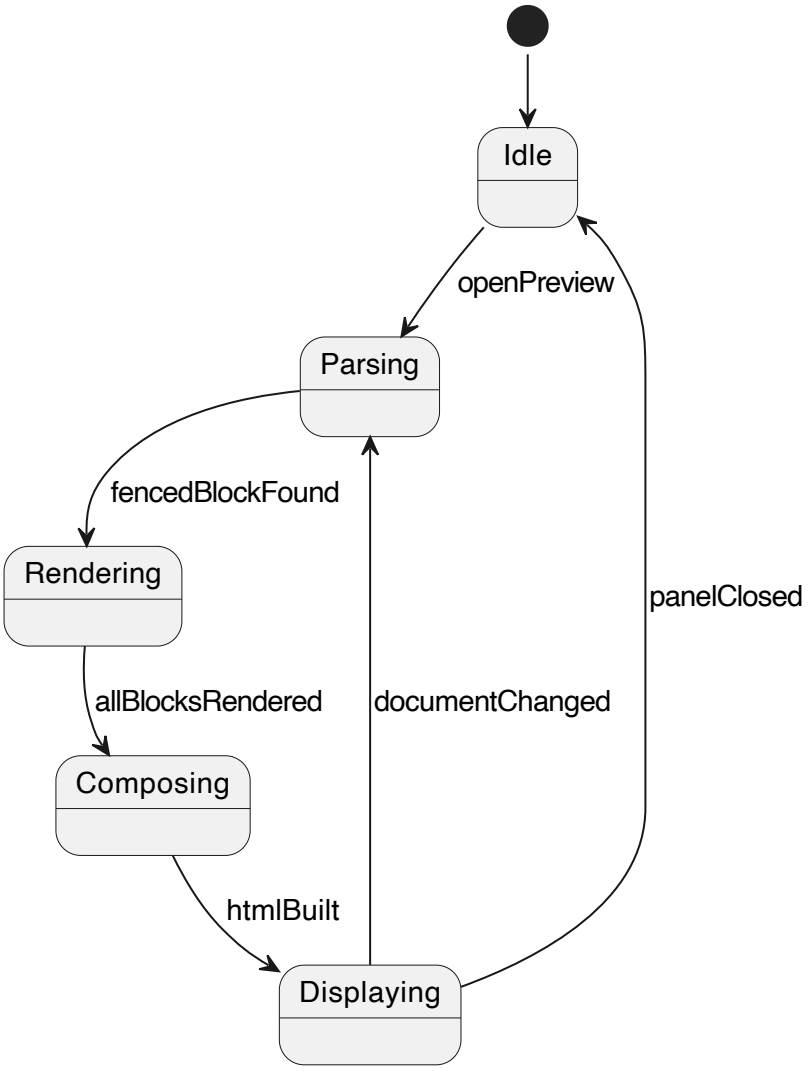
▼ Class Diagram



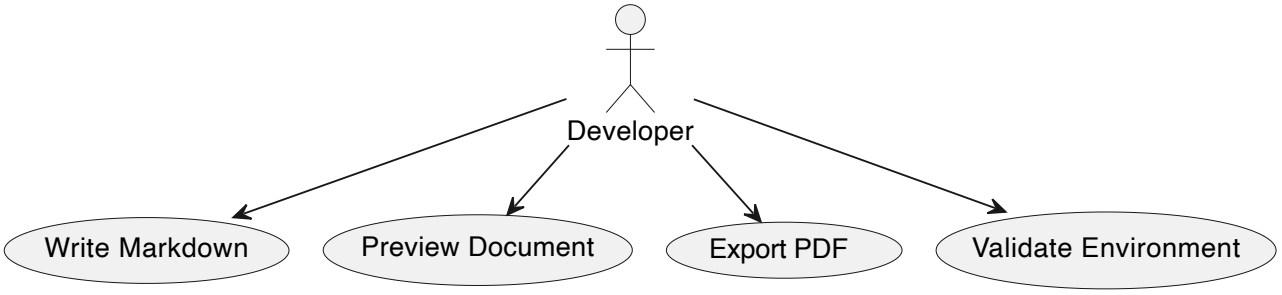
▼ Activity Diagram



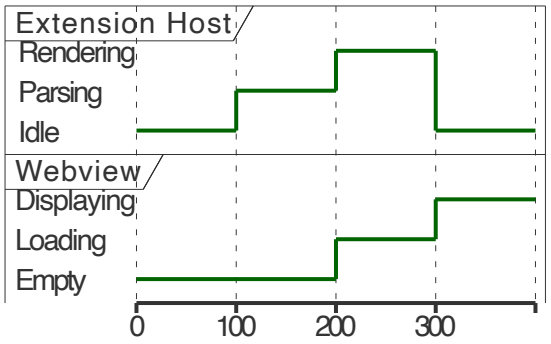

▼ State Diagram



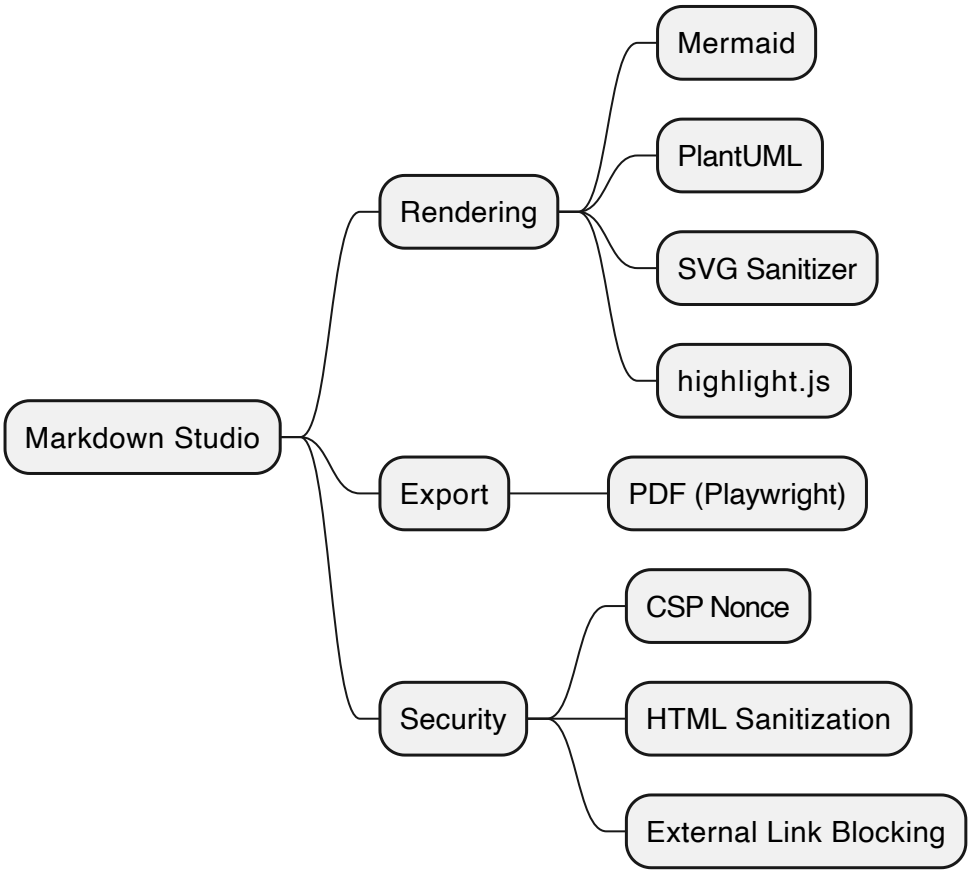

▼ Use Case Diagram



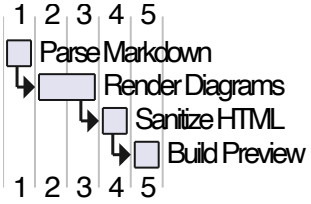

▼ Timing Diagram



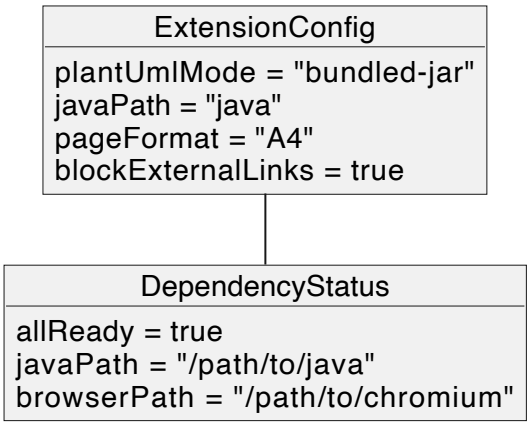

▼ Mind Map



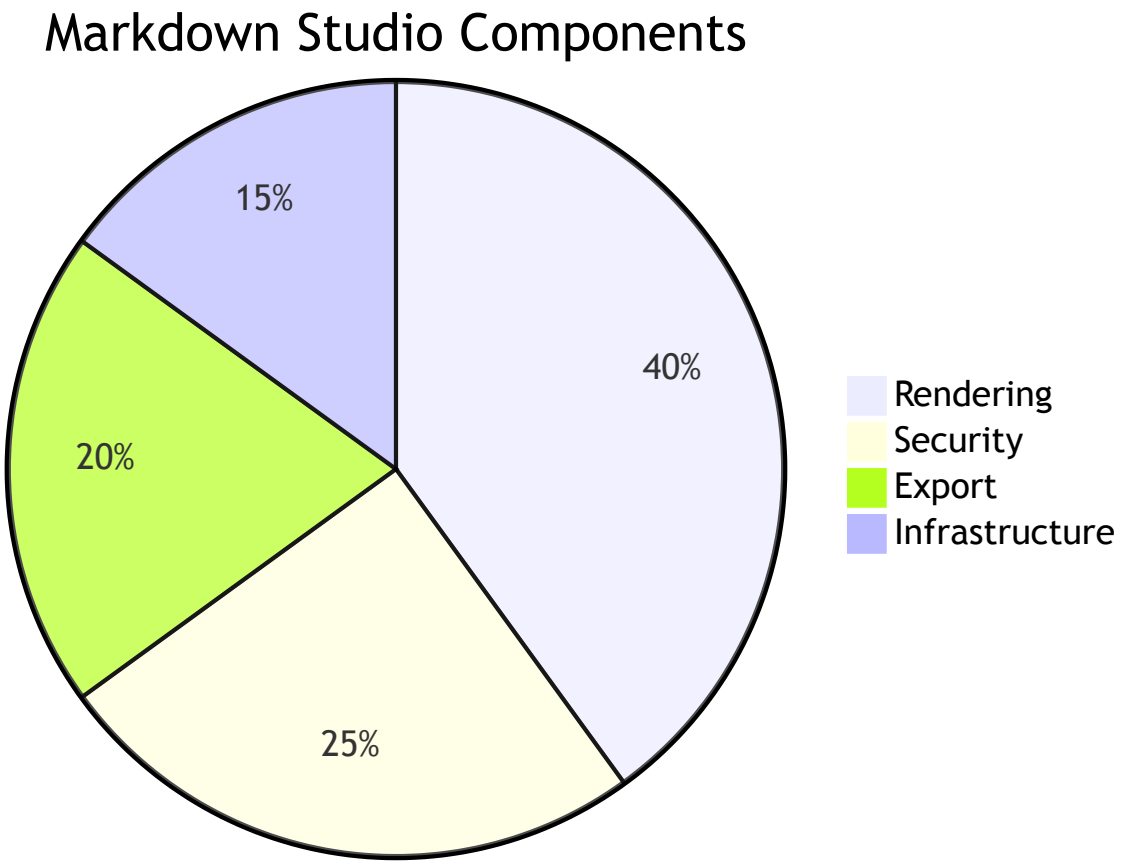

▼ Gantt Chart



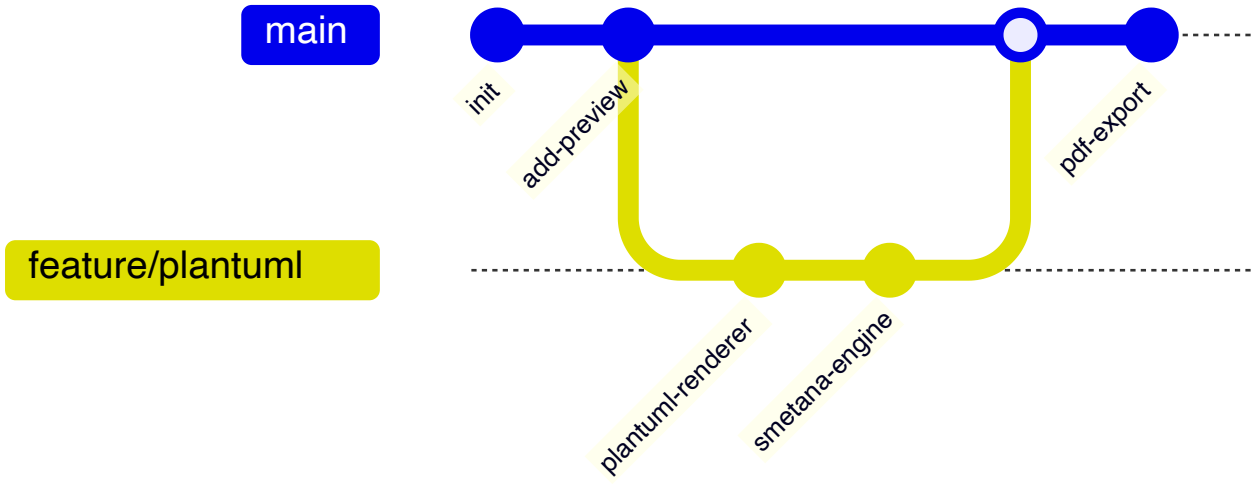

▼ Object Diagram



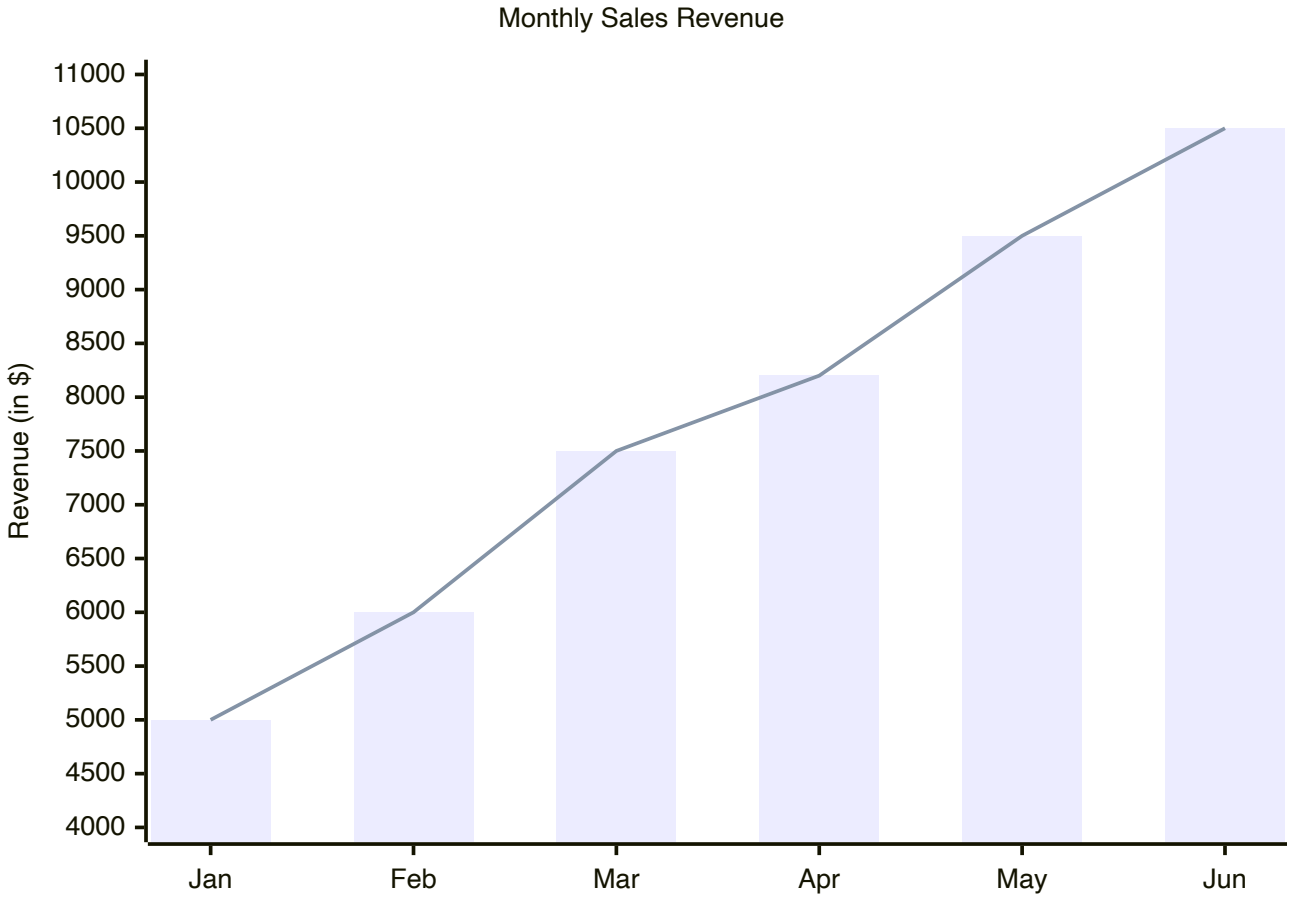

▼ Mermaid: Pie Chart



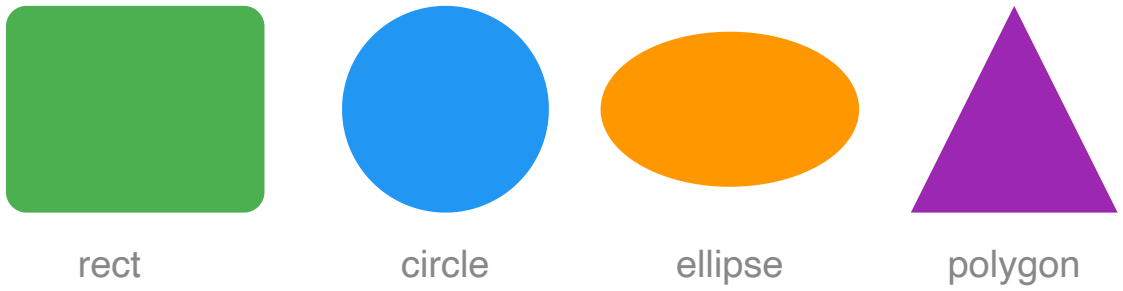

▼ Mermaid: Git Graph




▼ Mermaid: XY Chart



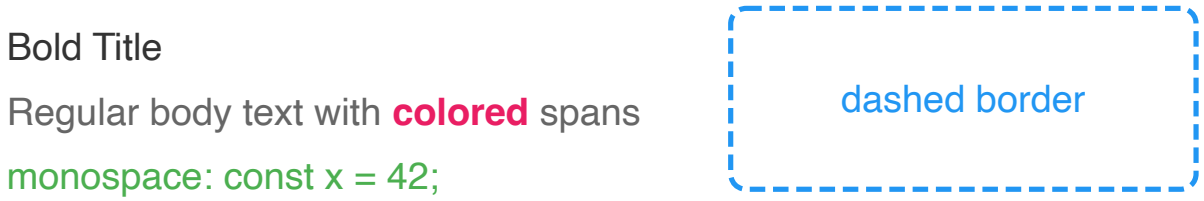

▼ SVG: Basic Shapes



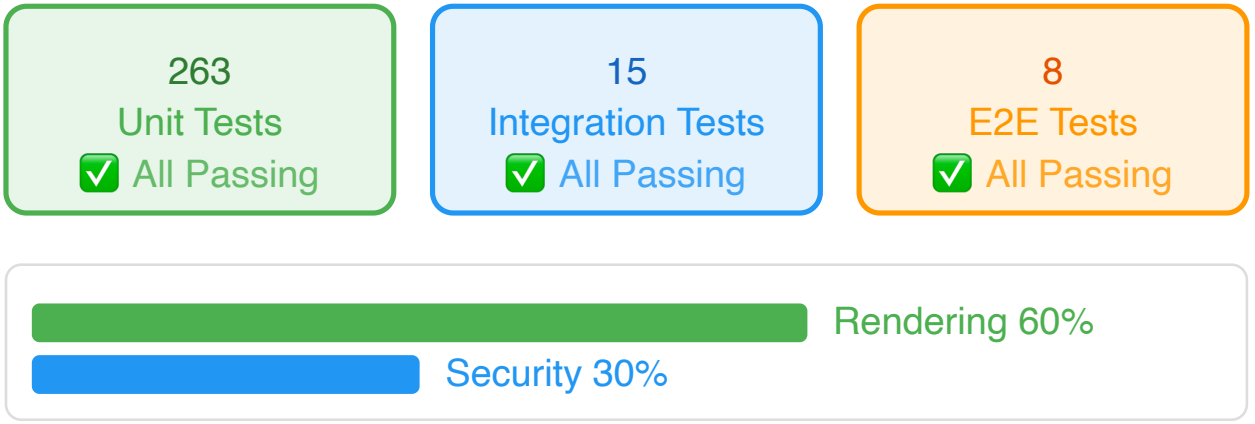

▼ SVG: Lines and Paths




▼ SVG: Text and Styling

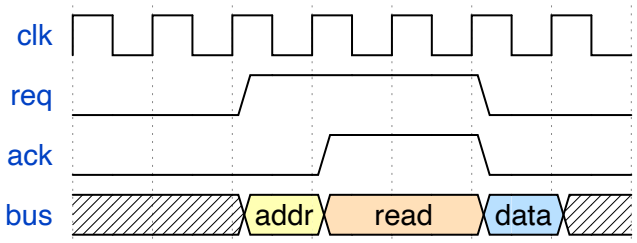



▼ SVG: Dashboard Layout

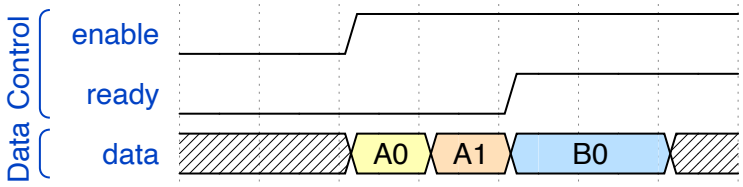


## WaveDrom Timing Diagram

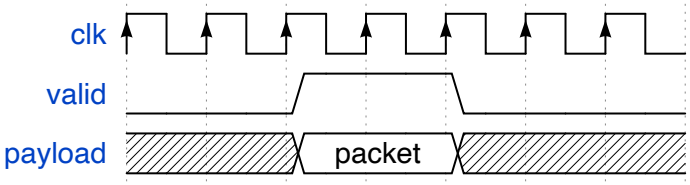
### Basic Handshake



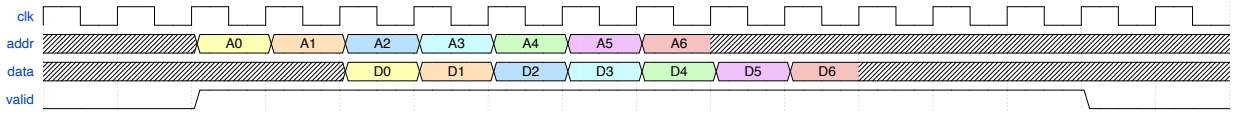

### Grouped Signals via wavejson Alias




### Strict JSON via wavedrom-json Alias




### Wide WaveDrom Bus




## Wide Diagrams for PDF Aspect Ratio

These examples are intentionally wider than the printable page. They verify that PDF export scales diagrams down without squashing their aspect ratio.

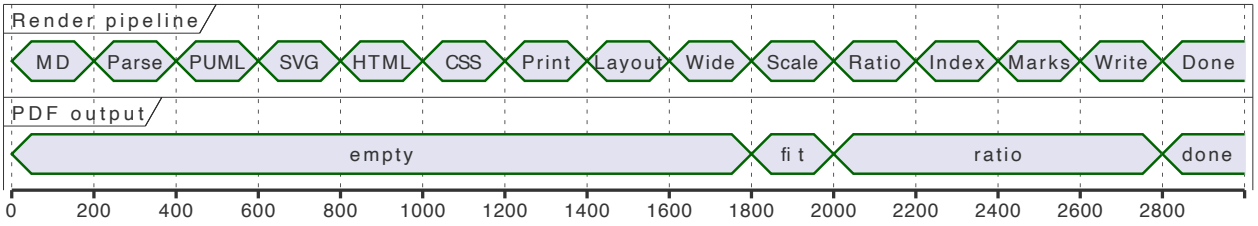
▼ Wide Mermaid flowchart




▼ Wide PlantUML component chain




▼ Wide PlantUML timing chart




▼ Wide SVG with fixed inline dimensions




---

## 9. Custom CSS

Markdown Studio supports built-in themes and inline CSS customization.

### Bundled Themes

Select a theme from the settings dropdown ( `markdownStudio.style.theme` ):

| Theme | Description |
|---|---|
| `default` | No extra styling — uses the selected preset only |
| `modern` | Indigo accents, soft shadows, refined typography |
| `markdown-pdf` | Classic look matching the popular Markdown PDF extension |
| `minimal` | Bare-bones, clean starting point |

All themes include dark mode support and print-optimized styles.

### Setup

```
// Pick a theme from the dropdown
"markdownStudio.style.theme": "modern"

// Optionally add your own CSS on top
"markdownStudio.style.customCss": "h1 { color: navy; } blockquote { border-left: 3px solid orange; }"
```

### How It Works

- Theme CSS is injected **after** the preset styles, so theme rules take priority
- `customCss` is applied **after** the theme, so your inline rules override everything
- The same CSS applies to both preview and PDF export for consistent output
- Copy any bundled theme and customize it to create your own

---

## 10. PDF Export

This entire document can be exported to PDF:

1. Open this file in VS Code
2. `Cmd+Shift+P` → **Markdown Studio: Export PDF**
3. A `demo.pdf` will be generated next to this file

The PDF uses the same HTML pipeline as the preview.

---

*Markdown Studio v0.8.0*

---

1. This is the first footnote content. ↩
2. This is the second footnote with **bold** text. ↩